

Agentic AI in Practice

From Spec to Productive Workflow

Tobias Weiß · DevOps, University of Marburg

GöAID Session · Academic Cloud

Where I come from: an operation, not a lab

- openEduSuite: sovereign groupware suite for teaching, research, administration — on bare-metal Kubernetes.
- **AI-infused** means two things here: LLM services *in* the suite, and agents working *on* the operation.
- Production, not a demo: real mailboxes, real students, real outages.

This session is the story of how specs turned into productive workflows.

Roadmap — 30 minutes, three blocks

Block	Content	Time
1 · Foundations	What "agentic" concretely means: model + harness + loop	~8 min
2 · Method	Spec → Contract → Agent → Verify: the pipeline	~10 min
3 · Practice	Four cases from the openEduSuite operation	~12 min

Afterwards: 10 minutes of questions.

Agentic ≠ chat

- Chat: question → answer. Agent: **goal** → **plan** → **tools** → **result** → **feedback**.
- Three building blocks: model + tools (terminal, files, git, cluster) + loop (plan, act, verify).
- The difference does not live in the model — it lives in the **harness** around it.

"Agentic" is not a model feature, it is a working environment.

The harness determines what the model can achieve

- Same model, different frame: chat window vs. terminal agent with file, git and cluster access.
- Our harness: **pi** — terminal agent, MCP tools, skills, memory across sessions.
- **Plan mode**: read and plan first, write later — the agent shows the plan before it touches anything.

What the agent may do is written in the contract — not left to gut feeling.

The pyramid: Spec · Contract · Test

- **Spec** — WHAT is to be built: requirement + acceptance criteria.
- **Contract** — WHAT the agent orients itself by: `AGENTS.md`, repo conventions, boundaries.
- **Test** — WHEN it is correct: CI, e2e suites, footer checks.

Every level is text in the repo — that makes work repeatable instead of heroic.

From spec to change: OpenSpec

- A **change** = proposal + spec delta + tasks — versioned in the repo, readable by human *and* agent.
- The agent implements task by task; the human reviews and **archives**.
- Archived means: the spec on main is the new state — **documentation is the product**, not a by-product.

```
Spec (OpenSpec) → Contract (AGENTS.md) → Agent (pi)
  → Verify (tests/CI) → Operate (ArgoCD)
```

The stage: openEduSuite on one slide

Component	Service	Component	Service
Identity	Keycloak (SSO)	Chat	Matrix/Synapse
Groupware	SOGGo	Projects	OpenProject
Files	OpenCloud	Video	Jitsi/Intercom
Wiki	XWiki	Portal	collab-dashboard

- GitOps: **ArgoCD** reconciles the cluster; bare metal (SCS), MariaDB/Galera, HAProxy.
- Agents work on this production system — hence: **spec before execution, test as acceptance**.

Case 1 · Domain migration of the whole suite — as a spec

- Starting point: cutover `home.openedu` → `suite.graphwiz.ai` — ingresses, certificates, OAuth redirects, mail domains across the whole stack.
- Human writes the **contract**: swap script idempotent, guard counters (157 + 8 manifests must *not* be touched), Keycloak redirects, Let's Encrypt wildcard.
- Agent implements `bootstrap-domain-swap.sh` — acceptance criteria green, only then execution.

| *A nerve-racking cutover becomes routine: you can simply run the script again.*

Case 2 · SSO mystery: white page after login

- Symptom: SOGo webmail shows a **white page** after SAML login — three root causes in a row.
- Agent in plan mode: hypotheses → probes (metadata dump, ACS URL, TLS hop at the ingress) → fix → test.
- Second case, same pattern: Stalwart XOAUTH2 broke on a dead OIDC issuer inside a directory object.

Debugging is the best agentic use case: symptom in, root-cause chain out — with a log you can reread.

Case 3 · "Fix-all": making drift visible

- Pattern: k8s directories with no ArgoCD app — tenant mail, LLM, XWiki; legacy from the imperative-apply era.
- Orders with acceptance criteria: every app under GitOps, every diff explained, nothing implicit.
- Side finding: `k8s/llm` — the model serving is part of the same suite.

Agents are great at the tedious work of whole directories — if the definition of done is in the contract.

Case 4 · An agent fleet with acceptance gates

- The SSO test suite (59 checks, 15 clients e2e) was built in batches: parallel workers, isolated git worktrees.
- Every task had exact acceptance criteria; merge only on green — otherwise the fleet runner discards it.
- Fleet = spec at scale: write contracts once, distribute many small orders.

Not one super-agent — many small orders with tests. Scalable, cheap, auditable.

What the operation learned

- **Tight specs over big contexts:** fewer tokens, same quality — the spec is the compressor.
- **Manage expectations:** agents do not deliver finished bots — they need contracts and tests.
- **Test first:** every incident becomes a test. Every warning in the suite is a documented decision.

Trust is good. Verification is cheaper.

The pattern, applied to your project

1. **Contract first:** `AGENTS.md` in the repo — what the agent may do, where the facts live.
2. **Spec change before code:** small tasks with acceptance criteria — proposal → tasks → apply → archive.
3. **CI is the acceptance:** green = done. The agent may repeat itself; you review the result.
4. **Memory across sessions:** decisions land in the repo and in memory — not in the chat history.

Start with one boring, important task. Not with the experiment "an agent replaces me".

Resources

- **openEduSuite** — `openedusuite.graphwiz.ai` · portal: `openedu.graphwiz.ai`
- **Workshop repo** (IT4Science Days 2026, 3h deck + exercises) —
GitHub/Codeberg: `IT4Science-Days-2026-agentic-ai-workshop`
- **Tools:** pi (terminal agent) · OpenSpec (spec workflow) · taskfleet (agent fleet) ·
ArgoCD (GitOps)

Everything is tangible: repos, specs, test suites — no slide magic.

Thank you!

Questions? — 10 minutes

Tobias Weiß · `tobias.weiss@uni-marburg.de`

GöAID · openEduSuite · University of Marburg